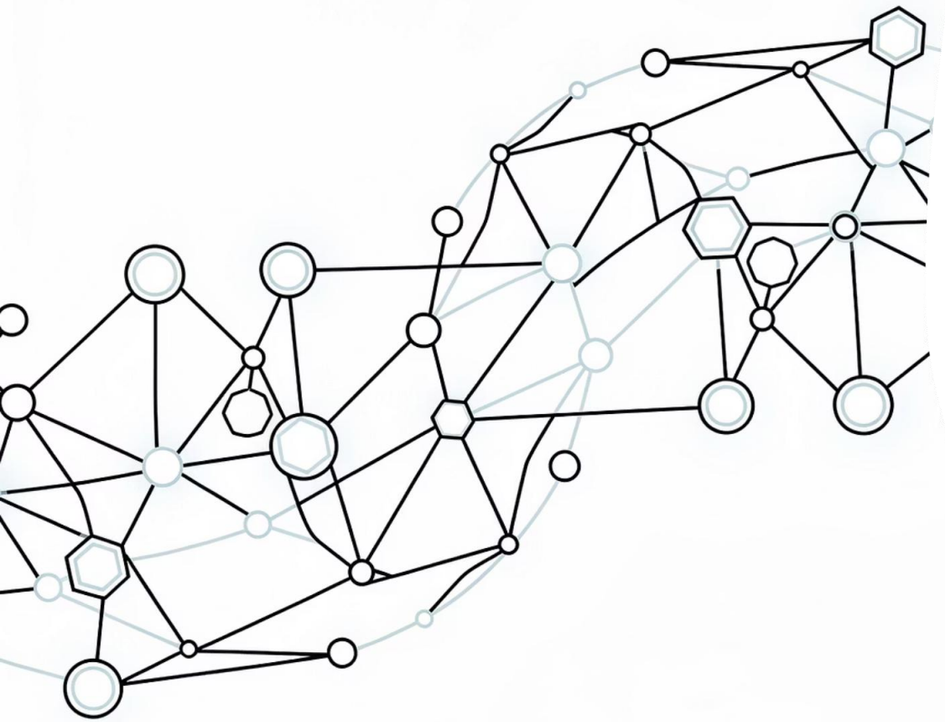


Neuronske Mreže

Od Osnova do PyTorch Implementacije

Sveobuhvatan vodič kroz arhitekturu neuronskih mreža, matematičke osnove i praktičnu implementaciju u PyTorch-u — od linearne regresije do klasifikacije.

Predmet: Duboko učenje i neuronske mreže, FON

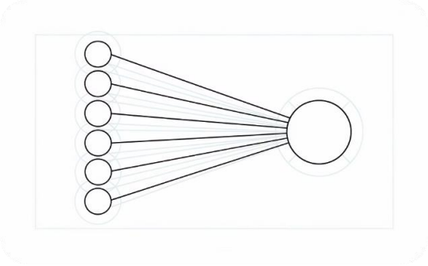
Prof dr Zoran Ševarac

Neuronska Mreža za Multinomijalnu Linearnu Regresiju

U regresijskom scenariju, cilj mreže je da predvidi **kontinuiranu numeričku vrednost** na osnovu skupa ulaznih promenljivih.

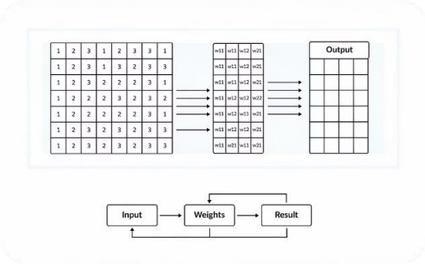
Primer iz prakse

Predviđanje cene kuće na osnovu površine (m^2), broja soba, lokacije i starosti objekta — svaki faktor je jedan ulazni neuron.



Arhitektura izlaza

Izlazni sloj ima tačno **jednu jedinicu** koja vraća predviđenu vrednost bez aktivacione funkcije — čista linearna kombinacija ulaza.



Matematička osnova

Izlaz se računa kao: `ŷ = X @ W + b`, gde su W težine a b pristrasnost. Minimizuje se srednja kvadratna greška (MSE).

Logistička Regresija sa Aktivacionom Funkcijom

Za razliku od linearne regresije, logistička regresija rešava problem **klasifikacije** — predviđa kojoj kategoriji pripada ulazni primer. Ključni dodatak je **aktivaciona funkcija** koja transformiše linearni izlaz u verovatnoću.

Sigmoidna funkcija

$$\sigma(x) = 1 / (1 + e^{-x})$$

sažima izlaz u opseg [0, 1], idealno za binarne klasifikacije.

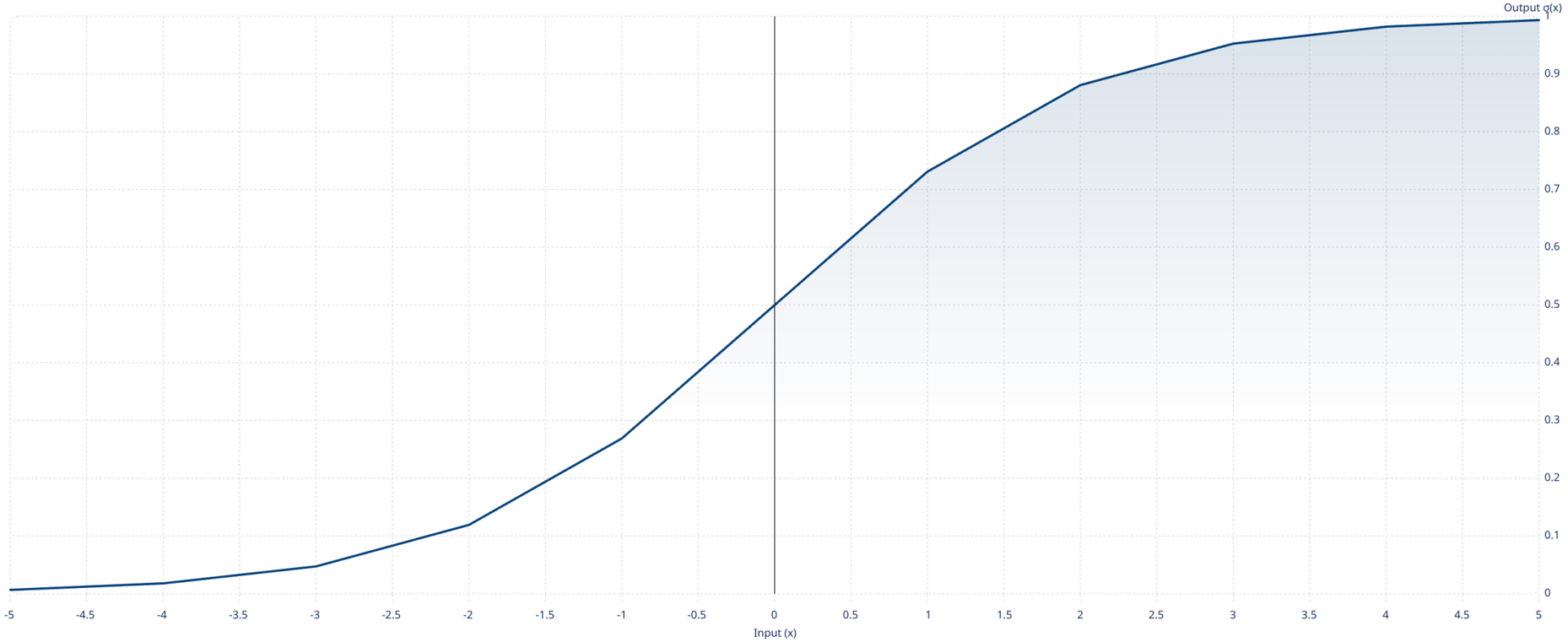
Primer klasifikacije

- Spam detekcija (spam / nije spam)
- Dijagnoza bolesti (pozitivan / negativan)

Izlazni sloj: **1 neuron** za binarnu klasifikaciju,

Sigmoidna Aktivaciona funkcija

The sigmoid function plays a crucial role in logistic regression, transforming any real-valued input into a value between 0 and 1. This output can be interpreted as a probability, making it suitable for binary classification tasks.



As shown in the graph, the sigmoid function produces an S-shaped curve that maps all real numbers to the range (0, 1). Inputs significantly larger than 0 are mapped close to 1, while inputs significantly smaller than 0 are mapped close to 0.

Ulazni i Izlazni Sloj

Ulazni sloj (Input Layer)

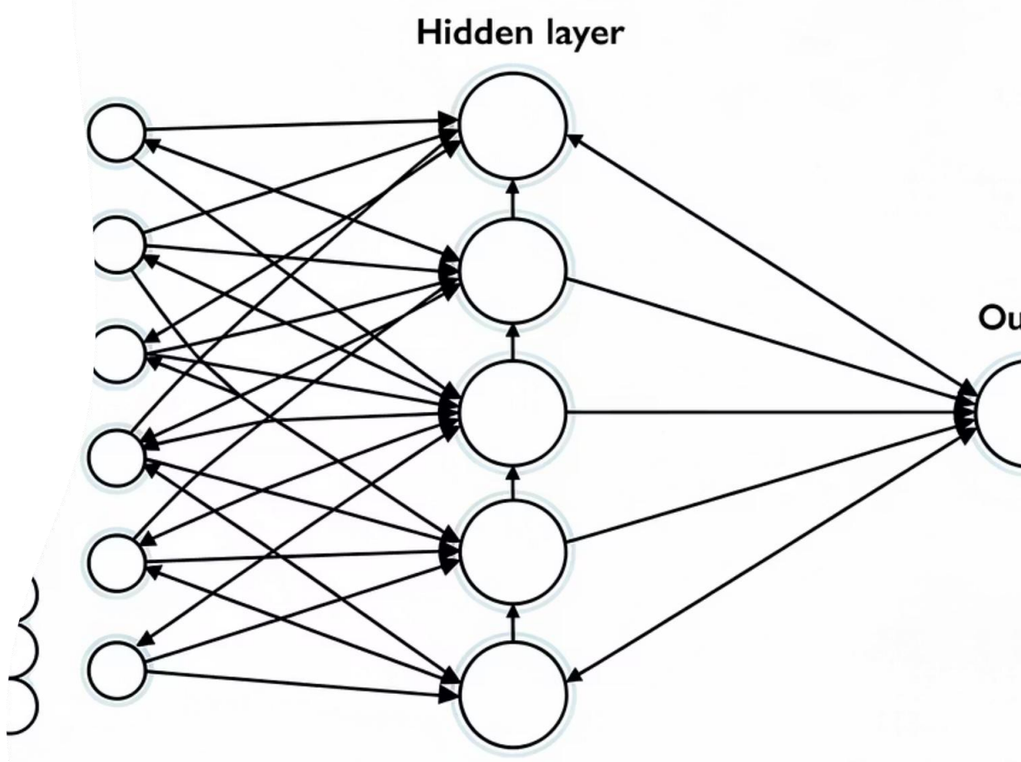
Prima sirove podatke iz spoljašnjeg sveta. Svaki neuron odgovara jednoj ulaznoj promenljivoj (feature-u). Ako imamo 5 feature-a, ulazni sloj ima **5 neurona**. Ovaj sloj ne vrši nikakvu transformaciju — samo prosleđuje podatke.

Skriveni slojevi (Hidden Layers)

Slojevi između ulaza i izlaza koji uče složene reprezentacije podataka. Svaki skriveni sloj primenjuje težinsku transformaciju i aktivacionu funkciju, izvlačeći sve apstraktnije feature-e.

Izlazni sloj (Output Layer)

Daje konačni rezultat modela. Broj neurona zavisi isključivo od zadatka: **1** za regresiju, **N** za N-klasnu klasifikaciju. Aktivaciona funkcija u ovom sloju određuje prirodu izlaza.



Neuronska Mreža kao Graf

Neuronska mreža se elegantno modeluje kao **usmereni aciklični graf (DAG)**. Ova reprezentacija čini strukturu mreže intuitivnom i vizuelno jasnom.

1 Čvorovi (nodes)

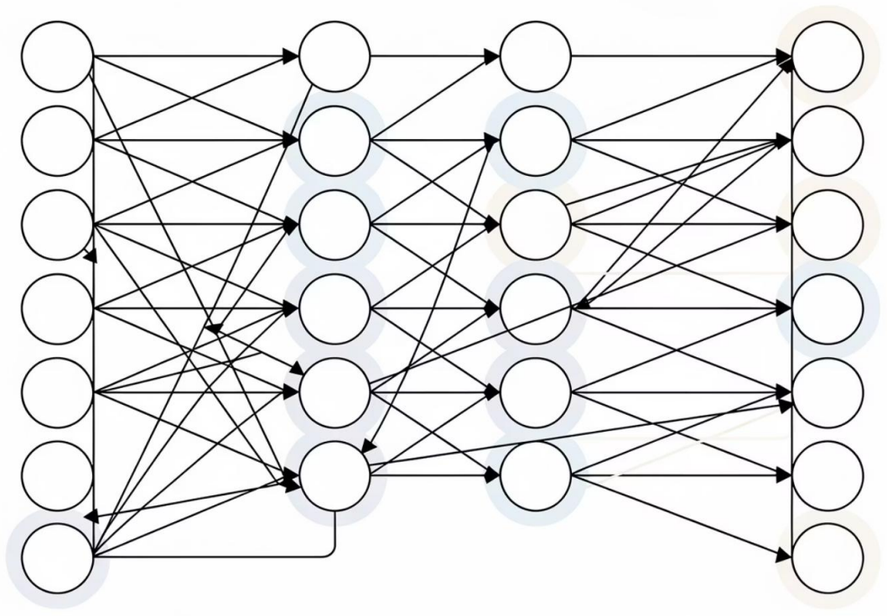
Svaki neuron u mreži predstavlja jedan čvor koji prima ulaze, vrši računanje i šalje izlaz dalje.

2 Usmerene grane (edges)

Veze između neurona nose težine (weights) koje određuju koliko jak uticaj jedan neuron ima na drugi.

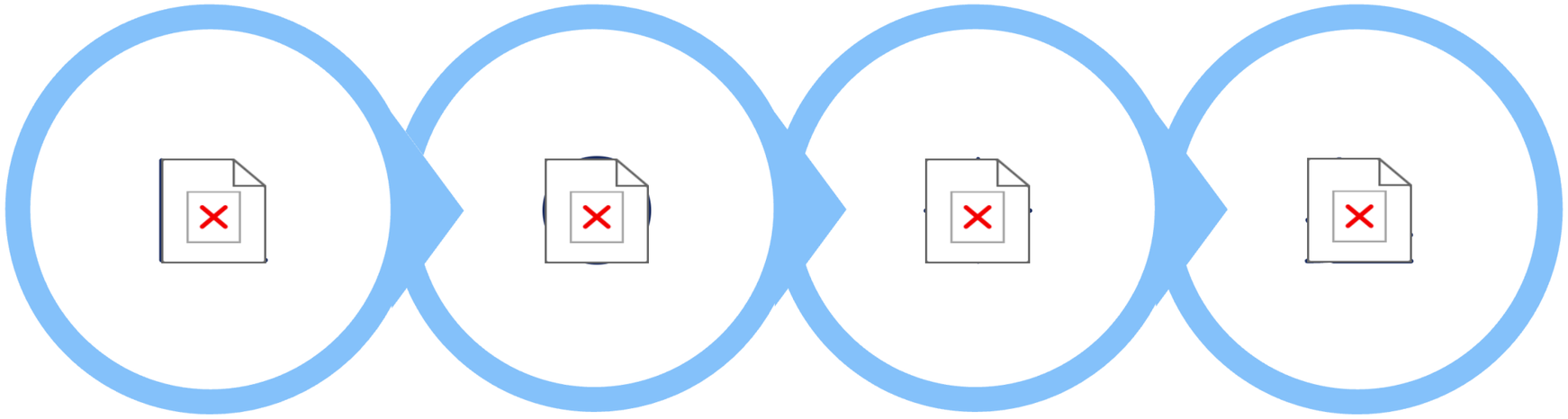
3 Tok podataka

Informacije teku isključivo u jednom pravcu — od ulaznog ka izlaznom sloju (forward pass).



Implementacija Pomoću Matrica: Blok Šema

Svaki sloj neuronske mreže može se kompaktno predstaviti kao jedna matrična operacija, što omogućava izuzetno efikasno paralelno izvršavanje na GPU hardveru.



Ulaz X | Množenje W | Dodaj b | Aktivacija f(z)

Ova reprezentacija je temelj svih modernih deep learning frejmvorka. Umesto petlji po neuronima, ceo sloj se izračunava jednom matričnom operacijom — što je i do hiljadu puta brže na odgovarajućem hardveru.

Mapiranje Konceptualnog Modela na Matrice

Ulazna Matrica X

Oblik: `(n_samples, n_features)` Svaki red je jedan uzorak podataka, svaka kolona je jedan feature. Na primer, 100 kuća sa 5 atributa → matrica oblika `(100, 5)`.

Matrica Težina W

Oblik: `(n_in, n_out)` Povezuje svaki neuron prethodnog sloja sa svakim neuronom trenutnog sloja. Ove vrednosti se uče tokom treninga kroz backpropagation.

Vektor Pristrasnosti b

Oblik: `(n_out,)` Dozvoljava modelu da pomeri aktivaciju nezavisno od ulaza. Ključan za fleksibilnost modela — bez njega mreža uvek prolazi kroz koordinatni početak.

Izlaz Sloja

Formula: `output = f(X @ W + b)` Gde je `f` aktivaciona funkcija. Ovaj izlaz postaje ulaz sledećeg sloja u mreži.

PyTorch: Ključne Komponente

Tensor

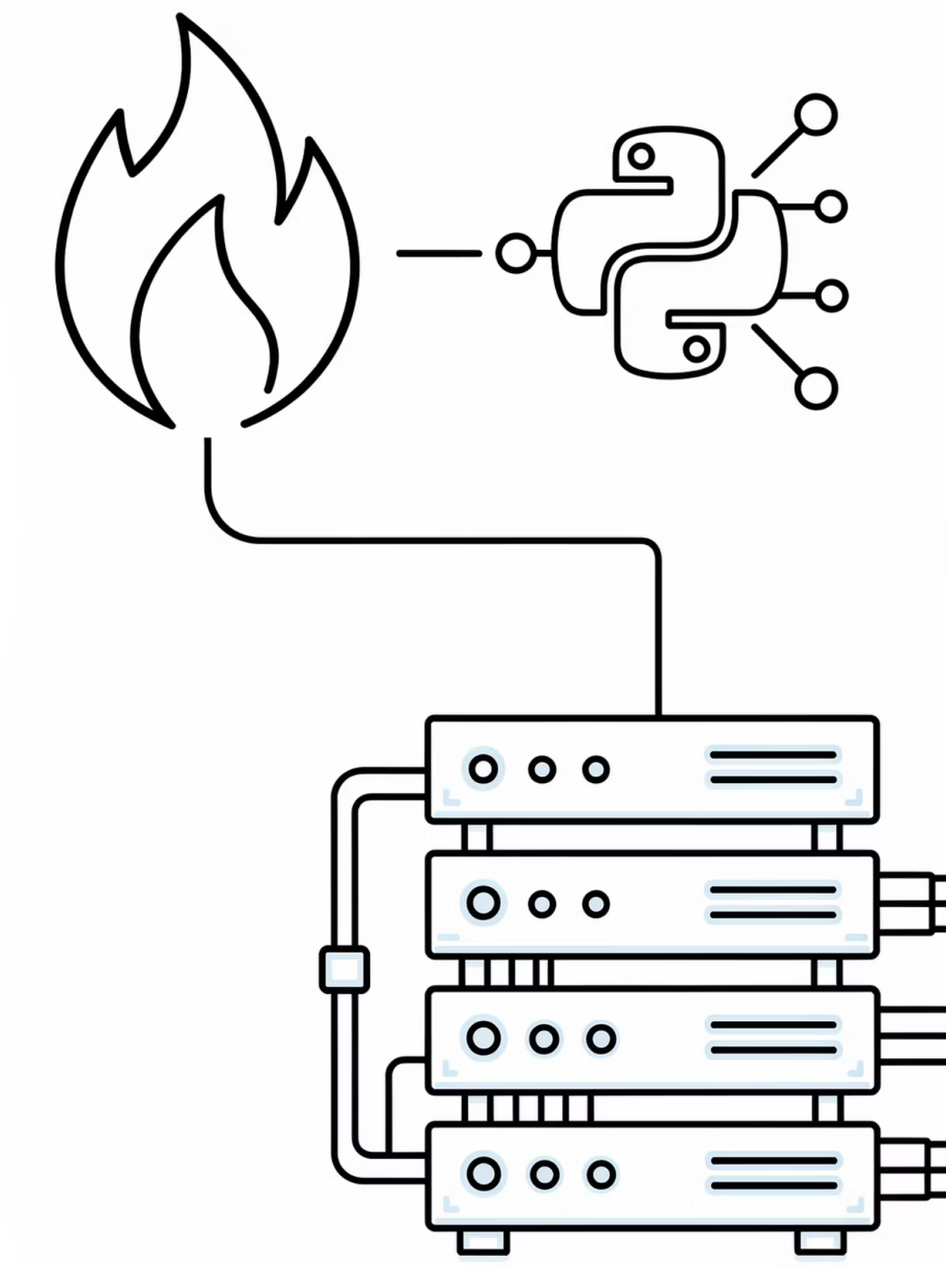
Osnovna struktura podataka u PyTorch-u. Slično NumPy `ndarray`, ali sa supermoćima: podrška za **GPU akceleraciju** i automatsko računanje gradijenata (`autograd`). Svi podaci i parametri modela su tenzori.

```
import torch
x = torch.tensor([[1.0, 2.0],
                  [3.0, 4.0]])
print(x.shape)  # torch.Size([2, 2])
```

Module

Apstraktna bazna klasa `nn.Module` za sve neuronske mreže. Svaki model ili sloj mora naslediti ovu klasu. Automatski prati parametre i podržava ugneždene module.

```
import torch.nn as nn
class MyNet(nn.Module):
 def __init__(self):
 super().__init__()
 self.fc = nn.Linear(4, 2)
 def forward(self, x):
 return self.fc(x)
```



PyTorch: `Sequential` Model

`nn.Sequential` je kontejner koji automatski povezuje slojeve redom — izlaz jednog sloja postaje ulaz sledećeg. Idealno za jednostavne **feedforward** arhitekture bez grananja.

Preglednost koda

Cela arhitektura je definisana u jednom bloku, čineći kod čitljivim i lakim za izmene.

Automatsko povezivanje

Nije potrebno ručno prosleđivati izlaz — Sequential to radi automatski u `forward` prolazu.

Ograničenja

Za složenije arhitekture sa grananjem ili višestrukim izlazima, koristi se custom `nn.Module` klasa.

Primer koda

```
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)

# Automatski forward pass:
output = model(input_tensor)
```

Ovaj model ima dva skrivena sloja (64 i 32 neurona) sa ReLU aktivacijama i jedan izlazni sloj sa jednom jedinicom.

PyTorch: `Linear` Sloj

`nn.Linear(in_features, out_features)` implementira potpuno povezanu (fully connected) linearnu transformaciju — svaki ulazni neuron je povezan sa svakim izlaznim neuronom.

Matematička operacija

Primenjuje: `y = x @ W.T + b` Težinska matrica W ima oblik `(out, in)`, a bias vektor b oblik `(out,)`. PyTorch automatski transponuje W.

Parametri sloja

`in_features`: broj ulaznih neurona
`out_features`: broj izlaznih neurona
Inicijalizuju se Kaiming uniform distribucijom po defaultu.

Broj parametara

Svaki Linear sloj ima: `in × out` težina + `out` bias vrednosti. Primer: `Linear(784, 128)` → 100,480 parametara ukupno.

```
layer = nn.Linear(in_features=784, out_features=128)
print(layer.weight.shape) # torch.Size([128, 784])
print(layer.bias.shape) # torch.Size([128])
```

PyTorch: ReLU Aktivaciona Funkcija

Rectified Linear Unit (ReLU) je najčešće korišćena aktivaciona funkcija u dubokim neuronskim mrežama. Jednostavna je, ali izuzetno efikasna u praksi.

Formula

$$\mathrm{ReLU}(x) = \max(0, x)$$

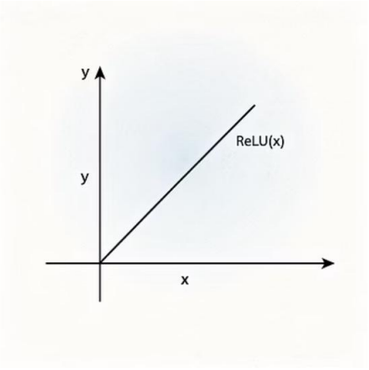
Sve negativne vrednosti postaju nula, pozitivne ostaju nepromenjene.

Zašto je važna?

Uvodi **nelinearnost** u model. Bez aktivacionih funkcija, cela mreža bi bila ekvivalentna jednoj linearnoj transformaciji, bez obzira na dubinu.

Prednosti ReLU-a

Brzo se računa, ne pati od problema nestajućih gradijenata kao Sigmoid, i empirijski daje odlične rezultate na većini zadataka.




```
activation = nn.ReLU()
x = torch.tensor([-2., -1., 0., 1., 2.])
print(activation(x))
# tensor([0., 0., 0., 1., 2.])
```

PyTorch: Funkcija Gubitka (Loss Function)

Funkcija gubitka je **kompas treninga** — meri koliko su predviđanja modela daleka od stvarnih vrednosti. Cilj treninga je minimizacija ove vrednosti.

nn.MSELoss

Mean Squared Error — koristi se za regresiju. Računa prosek kvadrata razlika između predviđenih i stvarnih vrednosti. Kažnjava velika odstupanja eksponencijalno.

```
loss_fn = nn.MSELoss()
loss = loss_fn(predictions, targets)
```

nn.CrossEntropyLoss

Unakrsna entropija — standardna funkcija gubitka za klasifikaciju. Kombinuje Softmax i negativni log-likelihood. Idealna za multinomijalnu klasifikaciju.

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, class_labels)
```

nn.BCELoss

Binary Cross-Entropy — koristi se za binarnu klasifikaciju kada izlazni sloj koristi Sigmoid aktivaciju. Svaki izlaz se tretira kao nezavisna binarna odluka.

```
loss_fn = nn.BCELoss()
loss = loss_fn(sigmoid_out, labels)
```

PyTorch: Optimizator (Optimizer)

Šta radi optimizator?

Optimizator implementira algoritam koji ažurira parametre (težine i bias) modela u suprotnom smeru gradijenata, smanjujući funkciju gubitka korak po korak.

```
optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

# Korak ažuriranja:
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Adam

Adaptivni optimizator koji prilagođava stopu učenja za svaki parametar ponaosob. Brz, robustan i de facto standard u dubokom učenju.

SGD

Stochastic Gradient Descent — klasičan pristup. Ažurira težine na osnovu gradijenta jednog mini-batch-a. Sa momentum parametrom postiže odlične rezultate.

RMSprop

Prilagođava stopu učenja za svaki parametar na osnovu pokretnog proseka kvadrata gradijenata. Posebno efikasan za rekurentne mreže.

Parametri Treninga: Epohe (Epochs)

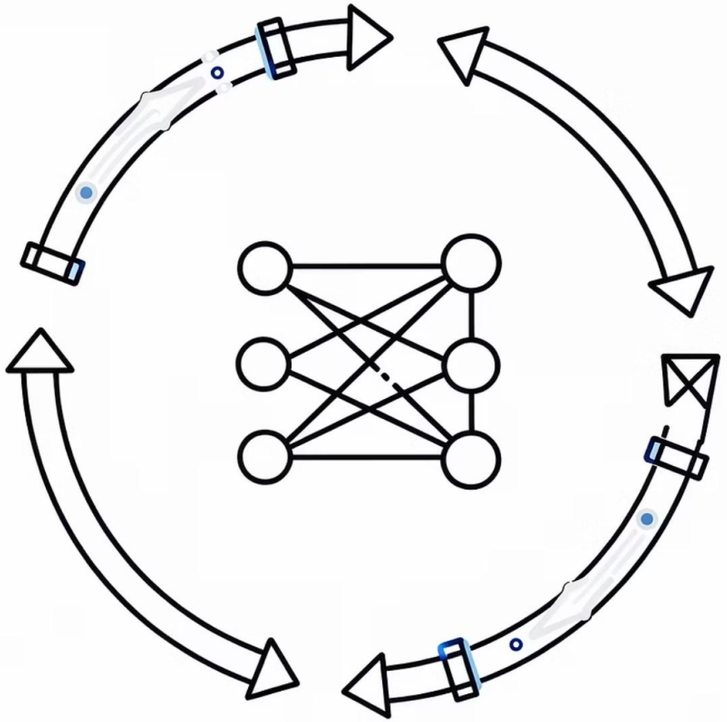
Šta je jedna epoha?

Jedna epoha predstavlja **jedan potpuni prolazak** kroz ceo trening skup podataka. Tokom jedne epohe, model vidi svaki trening primer tačno jednom i ažurira parametre na osnovu izračunatih gradijenata.

Koliko epoha je dovoljno?

Odgovor zavisi od kompleksnosti zadatka i veličine skupa podataka. Prerano zaustavljanje (underfitting) ili preterano treniranje (overfitting) su dva suprotna problema koja treba balansirati.

- **Premalo epoha** → model nije naučio obrasce (underfitting)
- **Previše epoha** → model je "zapamtio" trening podatke (overfitting)
- **Early stopping** → automatsko zaustavljanje kad val. greška počne da raste



Parametri Treninga: Stopa Učenja (Learning Rate)

Stopa učenja (`lr`) je jedan od **najvažnijih hiperparametara** u dubokom učenju. Kontroliše veličinu koraka kojim optimizator ažurira težine modela u svakom koraku.

0.001

Adam default LR

Najčešće korišćena početna vrednost za Adam optimizator

0.01

SGD tipičan LR

Uobičajena vrednost za SGD sa momentum parametrom

10x

LR Scheduler

Tipično smanjenje LR tokom treninga — svake N epoha

Primer: Neuronska Mreža za Regresiju u PyTorch-u

Kompletna implementacija

```
import torch
import torch.nn as nn
import torch.optim as optim

class RegressionNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(5, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )
    def forward(self, x):
        return self.net(x)

model = RegressionNet()
loss_fn = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(100):
    optimizer.zero_grad()
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
    loss.backward()
    optimizer.step()
```

Koraci implementacije

01

Definisanje arhitekture

Kreirati `nn.Module` klasu sa Linear slojevima i ReLU aktivacijama u `__init__` metodi.

02

Inicijalizacija loss-a i optimizatora

Izabrati `nn.MSELoss` za regresiju i `optim.Adam` kao optimizator sa željenom stopom učenja.

03

Petlja za trening

Za svaku epohu: nulirati gradijente, izračunati predikciju, izračunati loss, backpropagation, ažurirati težine.

04

Evaluacija

Pokrenuti model sa `model.eval()` i `torch.no_grad()` na test skupu da se izmeri generalizacija.

Vizuelizacija Rezultata Treninga

Praćenje gubitka tokom treninga

Ključni indikator uspešnosti treninga je **kriva gubitka** (loss curve). Idealno, train loss i validation loss se ravnomerno smanjuju i konvergiraju. Divergencija između njih signalizira overfitting.

Opadajuća greška / loss

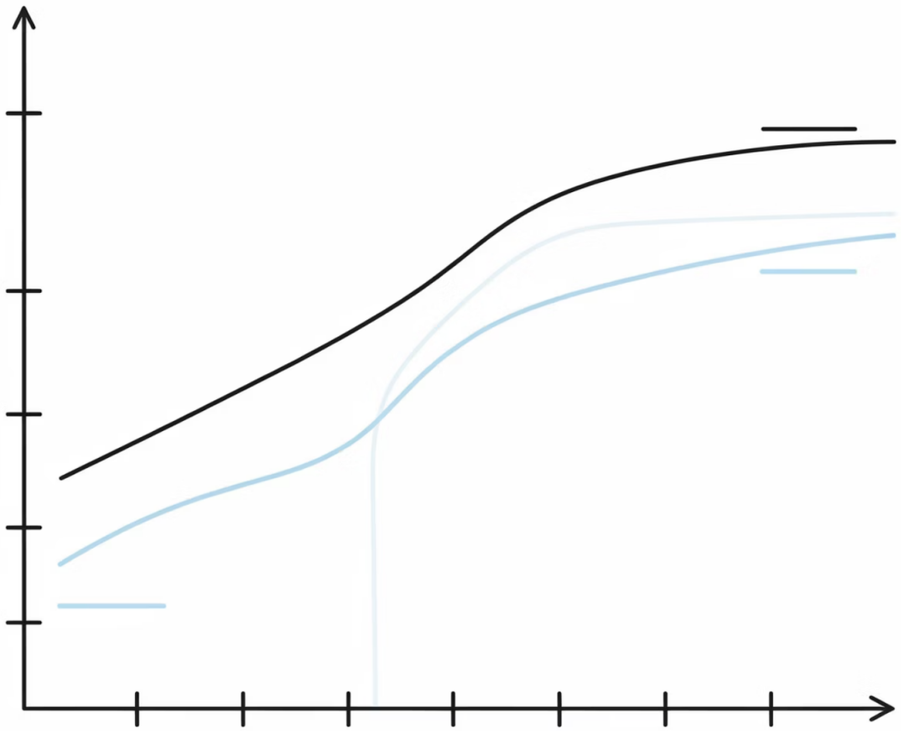
Model uči — gradijenti ažuriraju težine u pravom smeru

Usporena konvergencija (plateau)

Model je blizu minimuma — razmotrite smanjenje LR ili promenu arhitekture

Oscilacije

Stopa učenja je previsoka — smanjite `lr` za stabilniji trening



Od Linearne Regresije do Klasifikacije

Prelazak sa regresijskog na klasifikacijski model zahteva samo **nekoliko ključnih promena** — arhitektura ostaje suštinski ista, ali se menjaju izlazni sloj i funkcija gubitka.

1 Linearna Regresija

- Izlaz: 1 neuron (kontinuirana vrednost)
- Aktivacija izlaza: *bez aktivacije*
- Loss: `nn.MSELoss()`
- Metrika: MAE, RMSE, R^2

2 Binarna Klasifikacija

- Izlaz: 1 neuron (verovatnoća)
- Aktivacija: `nn.Sigmoid()` → [0, 1]
- Loss: `nn.BCELoss()`
- Metrika: Accuracy, F1, AUC-ROC

Zaključak: Moć Neuronskih Mreža

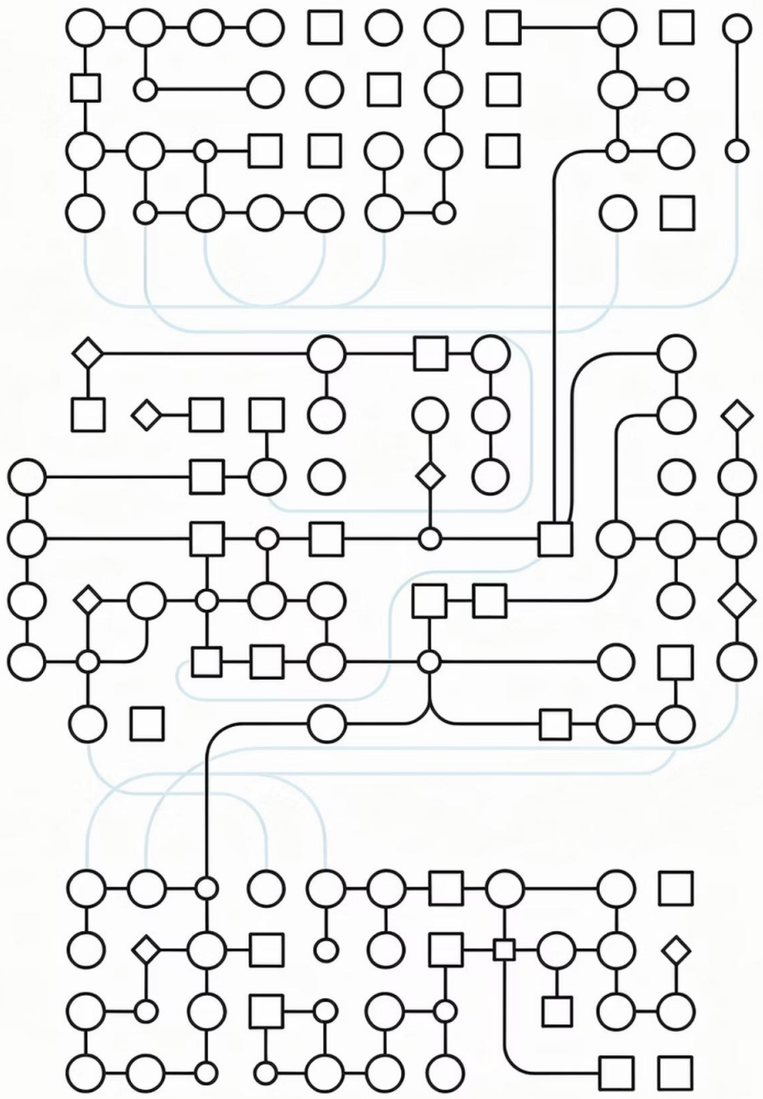
Univerzalni alat

Neuronske mreže su moćan, fleksibilan okvir koji se primenjuje na gotovo svaki problem u mašinskom učenju — od regresije i klasifikacije do generativnih modela.

PyTorch ekosistem

PyTorch pruža intuitivan, pythonski API za brzi razvoj prototipova i istraživanje, uz sve performanse potrebne za produkcionu implementaciju velikih modela.

Osnove su ključne

Razumevanje tenzora, slojeva, loss funkcija i optimizatora — gradivnih blokova — otvara vrata ka naprednim arhitekturama poput CNN, RNN i Transformer modela.

Sledeći koraci: Primenite naučene koncepte na realnom skupu podataka, eksperimentišite sa različitim arhitekturama i hiperparametrima, i istražite PyTorch dokumentaciju za napredne tehnike.